HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and Communications Technology

PROJECT REPORT

DEEP LEARNING – IT4653

Project

MovieRAG: Enhanced System with Movie Knowledge Graph

STT	Họ và Tên	MSSV	Email
1	Nguyễn Quốc Bảo Long	20230045	long.nqb230045@sis.hust.edu.vn
2	Nguyễn Đình Lâm Phúc	20235193	phuc.ndl235193@sis.hust.edu.vn
3	Nguyễn Khánh Hưng	20235100	hung.nk235100@sis.hust.edu.vn
4	Trần Nguyên Thành	20235224	thanh.tn235224@sis.hust.edu.vn

Supervisors

Assoc Prof. Nguyễn Hồng Quang
Class:

Ha Noi, 2026

Abstract

Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language understanding; however, they remain prone to hallucinations and lack up-to-date knowledge in specialized domains. While Retrieval-Augmented Generation (RAG) [1] mitigates these issues by retrieving relevant context, traditional vector-based approaches often treat data snippets in isolation, failing to capture the complex structural dependencies essential for reasoning. This paper presents the design and implementation of a movie-domain chatbot leveraging GraphRAG (Retrieval-Augmented Generation with Graphs) to address these limitations. Unlike naive RAG systems, our approach models the underlying movie database as a Knowledge Graph (KG) [2] , explicitly representing relationships between heterogeneous entities such as actors, directors, and genres. By integrating graph traversal algorithms with semantic retrieval, the system identifies and retrieves relevant subgraphs, providing the LLM with structured, topologically rich context for answer generation. Experimental evaluations demonstrate that the GraphRAG-based architecture [2] significantly outperforms baseline methods in handling multi-hop queries and relational reasoning tasks. This study highlights the efficacy of combining symbolic graph structures with neural language models to build more robust and context-aware conversational agents.

Contents

Chapter 1

Introduction

1.1 Motivation

The unprecedented evolution of Large Language Models (LLMs) has fundamentally redefined the boundaries of Natural Language Processing. These models have become ubiquitous, endowing artificial agents with the capacity to comprehend and generate human-like text with remarkable fluency [3], [4]. In the context of the entertainment industry, specifically movie recommendation and discovery, this advancement offers a significant opportunity to transform how users interact with cinematic databases. Users no longer seek simple keyword-based results; they demand intelligent agents capable of understanding complex, context-rich queries such as identifying artistic collaborations or exploring thematic similarities across film franchises.

However, the movie domain is characterized by high *relational density*. Entities such as actors, directors, genres, and production studios are not isolated data points but are inextricably linked through a complex web of artistic and professional collaborations. To fully leverage the potential of LLMs in this domain, there is a critical need for a framework that can not only retrieve information but also comprehend the **topological structure** of these relationships. This necessitates a paradigm shift towards Retrieval-Augmented Generation with Graphs (GraphRAG) [2], which utilizes Knowledge Graphs to preserve the semantic context necessary for answering sophisticated user queries.

1.2 Problem Statement

Despite their linguistic proficiency, deploying LLMs for specialized domain tasks presents significant challenges. The first intrinsic limitation is the phenomenon of hallucination [5], where the model generates plausible but factually incorrect assertions. Additionally, LLMs are constrained by their static training data, rendering them incapable of accessing real-time information or private knowledge bases developed after their training cutoff [1].

While conventional Retrieval-Augmented Generation (RAG) frameworks [1] attempt to mitigate these issues by indexing external knowledge within a vector database, they encounter critical bottlenecks:

1. **Loss of Structural Relationships:** Conventional RAG processes documents by partitioning them into discrete segments and encoding them into high-dimensional vectors. This "naive" vector-based approach treats data points as independent entities in a latent space, inadvertently discarding the explicit structural dependencies that exist between them.

2. **Inability to Handle Multi-hop Queries:** By flattening rich information into isolated vectors, vector-based systems struggle to synthesize information that requires traversing multiple logical steps. For instance, a query like *"Find movies directed by Christopher Nolan that feature actors who also played in Marvel films"* requires understanding the interconnected nature of entities, which is often beyond the reach of semantic similarity metrics like cosine similarity alone [2].

Therefore, the core problem this research addresses is the inadequacy of flat, vector-based retrieval methods in capturing the complex, interlinked semantics of the movie domain, and the need for a holistic GraphRAG [2] approach to enable accurate, context-aware reasoning.

1.3 Project Objectives

The primary goal of this project is to attempt to design and implement a **Graph Retrieval-Augmented Generation (GraphRAG)** [2] system tailored for the movie domain. This system aims to overcome the limitations of flat vector-based retrieval by integrating structured knowledge representations with the generative capabilities of Large Language Models.

To achieve this main goal, the project focuses on the following specific objectives:

1. **Construction of a Movie Knowledge Graph (MKG):** To design a comprehensive ontology and construct a Knowledge Graph using **Neo4j** [6], a widely-used tool for graph-based tasks. This involves modeling complex cinematic entities (e.g., *Director, Actor, Genre, Movie*) and their semantic relationships (e.g., *'DIRECTED_BY'*, *'ACTED_IN'*, *'SEQUEL_OF'*) to preserve the topological structure of the data that is typically lost in vector-based approaches.

2. **Development of a Graph-Enhanced Retrieval Pipeline:** To implement a retrieval mechanism that goes beyond simple semantic similarity [7], [8]. The system will utilize graph traversal algorithms (such as multi-hop reasoning) to extract interconnected subgraphs relevant to user queries, thereby addressing complex questions that require logical reasoning across multiple entities (e.g., finding common collaborators between two directors).

3. **Integration of Neuro-Symbolic AI:** To create a seamless integration between the symbolic reasoning of the Knowledge Graph and the probabilistic generation of LLMs (**Google Gemini**). This involves designing prompt templates that effectively utilize the retrieved graph context to ground the LLM's responses, minimizing hallucinations and ensuring factual accuracy.

4. **Empirical Evaluation and Comparison:** To evaluate the performance of the proposed GraphRAG framework against a baseline "Naive" RAG system (Vector-only).

The evaluation will focus on key metrics such as answer faithfulness, relevance, and the ability to correctly handle multi-hop queries.

1.4 Scope and Limitations

Scope of the Project

This research focuses on the development of a backend-heavy retrieval system and a user-facing chatbot interface. The specific boundaries of the project are defined as follows:

Data Coverage: The system utilizes the **TMDB (The Movie Database)** Dataset, where information of thousands of movies will be collected via their API then will be preprocessed. This dataset provides a rich collection of metadata, including plot summaries, cast and crew information, budget, revenue, and user ratings, which is sufficient to construct a dense Knowledge Graph for experimental purposes.

Functional Scope: The system supports two primary types of queries:

Recommendation Queries: "Suggest movies similar to Inception but with more horror elements."

Fact-based/Relational Queries: "Which director has collaborated the most with Leonardo DiCaprio?"

Technical Stack: The implementation is strictly limited to the integration of **Google Gemini** (via API) for language processing, **Qdrant** for vector database and **Neo4j** for graph storage.

Limitations

Despite the advantages of the GraphRAG approach, this project acknowledges several limitations due to constraints in resources and time:

1. **Linguistic Constraint:** The system is currently optimized for the **English language** only. While the underlying LLM (Gemini) supports multilingualism, the Entity Extraction and Graph Traversal pipelines are fine-tuned for English syntax. Queries in Vietnamese or other languages may result in suboptimal retrieval accuracy.

2. **Static Knowledge Base:** The constructed Knowledge Graph represents a **static snapshot** of the movie industry at the time of data collection. The system does not currently possess a real-time crawling mechanism to update the database with newly released movies or recent box-office figures.

3. **Cost inefficiency:** The utility of graph techniques requires an extensive amount of resources (hardware requirements, energy, money, ...). In this we can not afford that so our product should only be considered as an MVP (Minimum Viable Product) with integration of free-tier external services.

4. **Latency and API Dependency:** As the system relies on external API calls to Google Gemini for both embedding and generation, response latency is subject to network

stability and API rate limits. Therefore, the system is designed for proof-of-concept demonstrations rather than high-concurrency production environments.

Chapter 2

Background and Theoretical Framework

This chapter establishes the theoretical framework for the retrieval systems used in this research. We begin by defining the general Retrieval-Augmented Generation (RAG) paradigm. We then analyze the traditional Vector-based approach (Dense Retrieval) and its limitations regarding structural reasoning. Finally, we introduce Knowledge Graphs and the GraphRAG architecture as a solution for capturing complex relational dependencies.

2.1 Related Work

The evolution of information retrieval and recommendation systems in the entertainment domain can be categorized into three distinct phases: Traditional Collaborative Filtering, LLM-based Generation, and Retrieval-Augmented approaches.

2.1.1 Traditional Recommendation Systems

Early research in this domain was dominated by Collaborative Filtering (CF) and Matrix Factorization techniques [9]. These methods operate on the assumption that users who agreed in the past will agree in the future.

Strength: Effective at identifying latent user preferences based on historical interaction data (clicks, ratings).

Limitation: They suffer from the well-known "cold-start" problem (inability to recommend to new users) and the "semantic gap"—they cannot understand the nuances of natural language queries (e.g., *"A movie about loneliness in space"*).

2.1.2 LLMs as Direct Recommenders

With the emergence of Large Language Models (LLMs) like GPT-3 [10] and [11], [12], researchers began exploring zero-shot recommendation capabilities [4]. By prompting the model directly, users can receive fluent recommendations.

Strength: Superior natural language understanding and the ability to generate explanations.

Limitation: As highlighted by Ji et al. [13], these systems are prone to **hallucinations** (inventing non-existent movies or casting details). Furthermore, due to the static nature of their training sets, they cannot provide information on movies released after their knowledge cutoff date.

2.1.3 Vector-based Retrieval-Augmented Generation (RAG)

To mitigate hallucinations, Lewis et al. [1] introduced the RAG framework, which retrieves relevant documents from an external vector database to ground the LLM. This primitive RAG approach typically uses dense vector embeddings (e.g., Sentence-BERT [8]) to match query semantics with document chunks.

Strength: Significantly improves factual accuracy and allows for real-time knowledge updates.

Limitation: Vector-based approaches treat data points as independent entities in a high-dimensional space. As noted in recent GraphRAG research [2], they struggle with multi-hop reasoning tasks that require traversing complex relationships (e.g., connecting a director to a genre through a specific actor). This limitation is particularly pronounced in the movie domain, which is inherently a dense network of relationships.

The identified limitations in vector-based methods necessitate a shift towards **Graph-based RAG**, which explicitly models relationships to enable deeper reasoning and more accurate context retrieval.

2.2 Retrieval-Augmented Generation (RAG) Framework

2.2.1 Paradigm Overview

Retrieval-Augmented Generation (RAG), first formalized by Lewis et al. [1], represents a hybrid architectural paradigm that synergizes the generative capabilities of Large Language Models (LLMs) with the precision of Information Retrieval (IR) systems.

Fundamentally, RAG addresses the limitations of standard LLMs by decoupling the knowledge storage from the reasoning capability. In a traditional LLM (e.g., GPT-4), all knowledge is implicitly stored within the model's weights (referred to as **Parametric Memory**). While efficient, this memory is static, computationally expensive to update, and prone to hallucinations [5] when the model is unsure.

Conversely, RAG introduces an external, differentiable knowledge source (referred to as **Non-Parametric Memory**), typically implemented as a vector database. The workflow operates in two distinct phases:

1. **Retrieval Phase:** Given an input query x, the system searches the non-parametric memory to identify a set of relevant documents, or uses appropriate tools for the context gotten from the query x.

2. **Generation Phase:** The retriever passes both the original query x and the retrieved context Z to the generator (LLM). The model then generates the final response y by

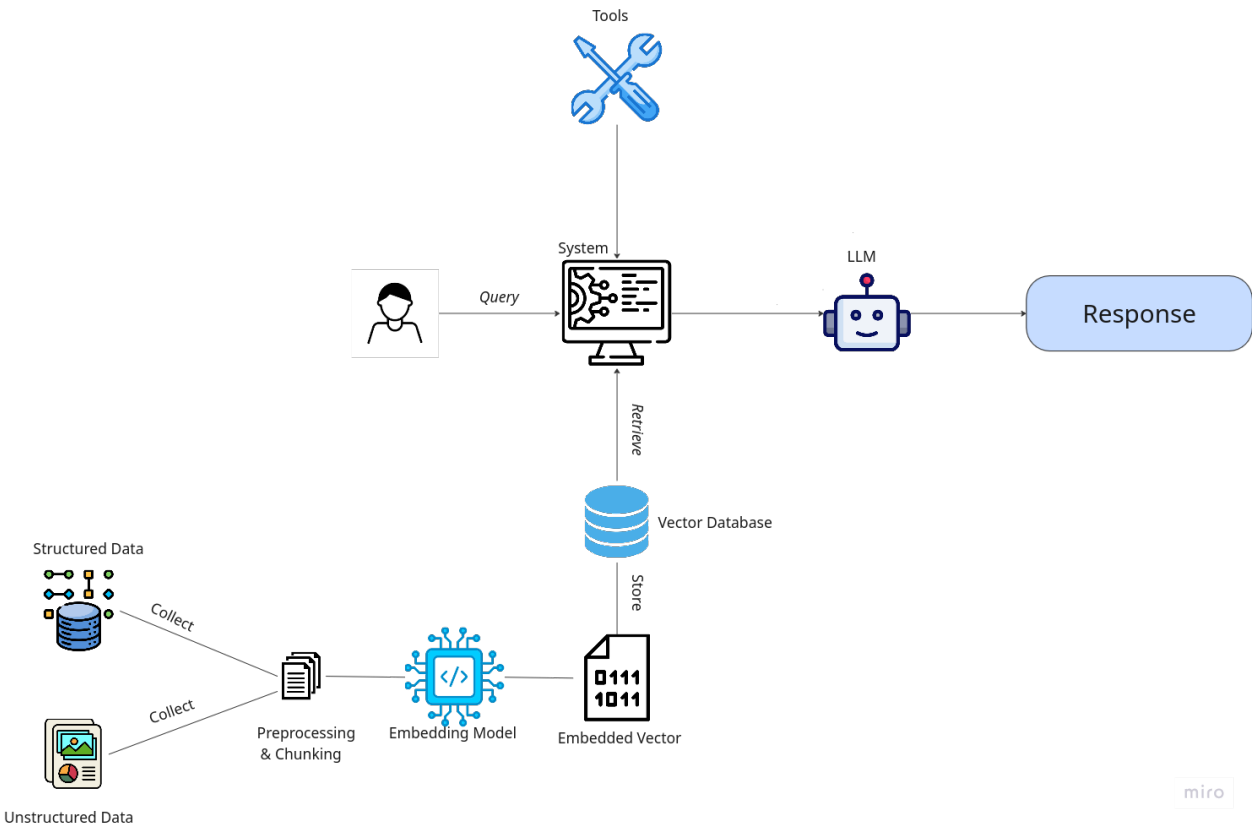

Figure 2.1: A RAG pipeline

maximizing the probability $P(y|x, Z)$.

This allows the generation process to be grounded in external evidence, which makes the response more concise and relevant.

2.2.2 The Role of External Knowledge

In the context of information-intensive tasks, relying solely on the pre-trained knowledge of LLMs presents significant challenges. The integration of external knowledge sources serves three critical functions within the RAG framework:

1. **Mitigating Knowledge Cutoffs (Temporal Agility):** LLMs are trained on static datasets collected up to a specific point in time. Consequently, they are *blind* to events or data generated post-training. In the movie domain, where new films are released weekly and ratings fluctuate in real-time, this limitation is critical. External knowledge bases allow the system to access the most current information without the prohibitive computational cost of continuous model retraining (Fine-tuning).

2. **Domain Specificity and Granularity:** While foundation models possess broad general knowledge, they often lack depth in specialized domains. An external database ensures the model has access to precise, granular metadata—such as specific production budgets, obscure cast members of indie films, or exact revenue figures—that may be underrepresented or "forgotten" in the model's vast parametric memory.

3. **Explainability and Trust:** When an LLM generates an answer purely from its weights,

the information source is opaque (a "black box"), making it impossible to verify. By explicitly retrieving documents or graph nodes, RAG enables "source tracking." The system can provide citations (e.g., *"According to the TMDB record for 'Inception'..."*), thereby allowing users to verify the accuracy of the recommendations and reducing the risk of confident hallucinations.

2.3 Vector-Based Retrieval (Dense Retrieval)

2.3.1 Semantic Embeddings

In a Retrieval-Augmented Generation system, the retrieval engine must bridge the gap between natural language queries and stored documents. Traditional keyword-based methods (e.g., TF-IDF [14]) rely on exact lexical overlapping, which fails to capture synonyms or contextual meaning. To overcome this, we employ Dense Semantic Embeddings.

Concept

Semantic embedding is the process of transforming text input (words, sentences, or paragraphs) into a fixed-size vector of real numbers in a high-dimensional space. Formally, given an embedding function f_θ and a text sequence S, the output is a vector v:

$$v = f_\theta(S) \in \mathbb{R}^d \tag{2.1}$$

where d is the dimensionality of the vector space (typically 384, 768, or 1536 depending on the model).

In this latent space, the geometric distance between two vectors corresponds to their semantic similarity. For instance, in the movie domain, the vector for *"Interstellar"* will be closer to *"Space exploration"* than to *"Romantic comedy"*, even if the words "space" or "exploration" do not appear explicitly in the movie's title.

Sentence-BERT (SBERT)

While models like BERT [3] revolutionized NLP, they are computationally expensive for retrieval tasks because they require feeding both sentences into the network simultaneously (Cross-Encoder).

The Sentence-BERT (SBERT) [15], a modification of the BERT network that uses **Siamese Networks** to derive semantically meaningful sentence embeddings. SBERT processes sentences independently to generate fixed vectors, allowing for highly efficient comparison using Cosine Similarity. This architecture enables the pre-computation (indexing) of millions of movie descriptions, making real-time retrieval feasible.

Gemini Embedding models [16]

Semantic embedding is the process of transforming text input (words, sentences, or paragraphs) into a fixed-size vector of real numbers in a high-dimensional space. Formally, given the Gemini embedding function E_{Gemini} and a text sequence S, the output is

a dense vector v:

$$v = E_{Gemini}(S) \in \mathbb{R}^d \tag{2.2}$$

where d represents the dimensionality of the vector space (typically $d = 768$ for standard Gemini embedding models).

In this latent space, the geometric distance between two vectors corresponds to their semantic similarity. For instance, the vector for *"The Dark Knight"* will be spatially closer to *"Crime thriller"* than to *"Romantic comedy"*, enabling the system to understand content beyond mere keyword matching.

Gemini Embedding Architecture

Unlike traditional open-source models (e.g., Word2Vec [17]) which generate static embeddings, Gemini utilizes a modern Transformer-based architecture [18] optimized for semantic retrieval tasks. Key advantages utilized in this project include:

- **Task-Specific Embeddings:** The Gemini API distinguishes between *retrieval_query* (the user's question) and *retrieval_document* (the knowledge base content). This asymmetry allows the model to optimize the vector space specifically for matching questions to answers, rather than just matching similar sentences.
- **Multilingual Support:** Trained on a massive, diverse corpus, Gemini embeddings demonstrate superior performance in capturing nuances across different languages, which is crucial for handling movie titles and descriptions that may contain international context.
- **High Dimensionality:** With a vector dimension of 768, the model captures a richer set of semantic features compared to smaller models (e.g., 384 dimensions), resulting in higher precision during the retrieval phase.

2.3.2 Similarity Measures

Information retrieval relies on measuring the distance between the query vector q and document vector d. The most common metric is Cosine Similarity:

$$\text{CosSim}(q, d) = \frac{q \cdot d}{\|q\| \|d\|} = \frac{\sum_{i=1}^{n} q_i d_i}{\sqrt{\sum_{i=1}^{n} q_i^2} \sqrt{\sum_{i=1}^{n} d_i^2}} \tag{2.3}$$

2.3.3 Approximate Nearest Neighbor (ANN) Search

In a real-world scenario with a large corpus of movie data, calculating the Cosine Similarity between the query vector and *every* stored vector (Exact k-NN) is computationally prohibitive. This "brute-force" approach has a time complexity of $O(N \cdot d)$, where N is the number of documents and d is the vector dimension. To ensure low-latency retrieval, this system employs Approximate Nearest Neighbor (ANN) algorithms.

The Trade-off: Precision vs. Speed

ANN algorithms relax the guarantee of finding the absolute "closest" neighbor in exchange for a significant reduction in search time (typically achieving logarithmic com-

plexity $O(\log N)$). The system organizes vectors into an index structure that allows the retrieval engine to traverse the vector space efficiently, inspecting only a small subset of candidate vectors that are likely to be relevant.

Hierarchical Navigable Small World (HNSW)

For this project, we utilize the HNSW algorithm [19], which is currently the state-of-the-art method for vector indexing (used by default in vector databases like Qdrant and Neo4j).

HNSW constructs a multi-layered graph structure:

Upper Layers (Express Lanes): Contain sparse connections (long-range links) allowing the search algorithm to quickly "zoom in" on the general neighborhood of the query vector.

Lower Layers (Local Roads): Contain dense connections for fine-grained traversal to identify the specific nearest neighbors.

This hierarchical structure mimics the "six degrees of separation" phenomenon, allowing the system to locate the most relevant movie vectors among thousands of entries in milliseconds.

2.3.4 Limitations in Multi-hop Reasoning

While vector-based retrieval excels at finding semantically similar documents, it encounters a fundamental bottleneck when handling Multi-hop Queries—questions that require retrieving information from multiple distinct documents and chaining them together logically.

The *Information Flattening* Problem

Vector embedding models compress complex structured data (entities and relationships) into a single flat vector in a latent space. In this process, the explicit structural dependencies between entities are often lost or flattened.

- **Vector Proximity $\neq$ Logical Relationship:** Two vectors might be close in space because they share similar keywords (e.g., "Hollywood", "Cinema"), not because there is a verified factual link between them.
- **Loss of Transitivity:** If Fact A is in Document 1 and Fact B is in Document 2, a vector search engine treats them as independent. It lacks the mechanism to perform transitive reasoning (i.e., if $A \to B$ and $B \to C$, then $A \to C$).

Case Study: The "Tom Cruise" Example

Consider the complex query: *"Who is the ex-wife of Tom Cruise that starred in a horror movie?"*

1. **Vector Search Failure:** A vector-based system will embed this query and look for documents containing terms like "Tom Cruise", "Ex-wife", and "Horror".

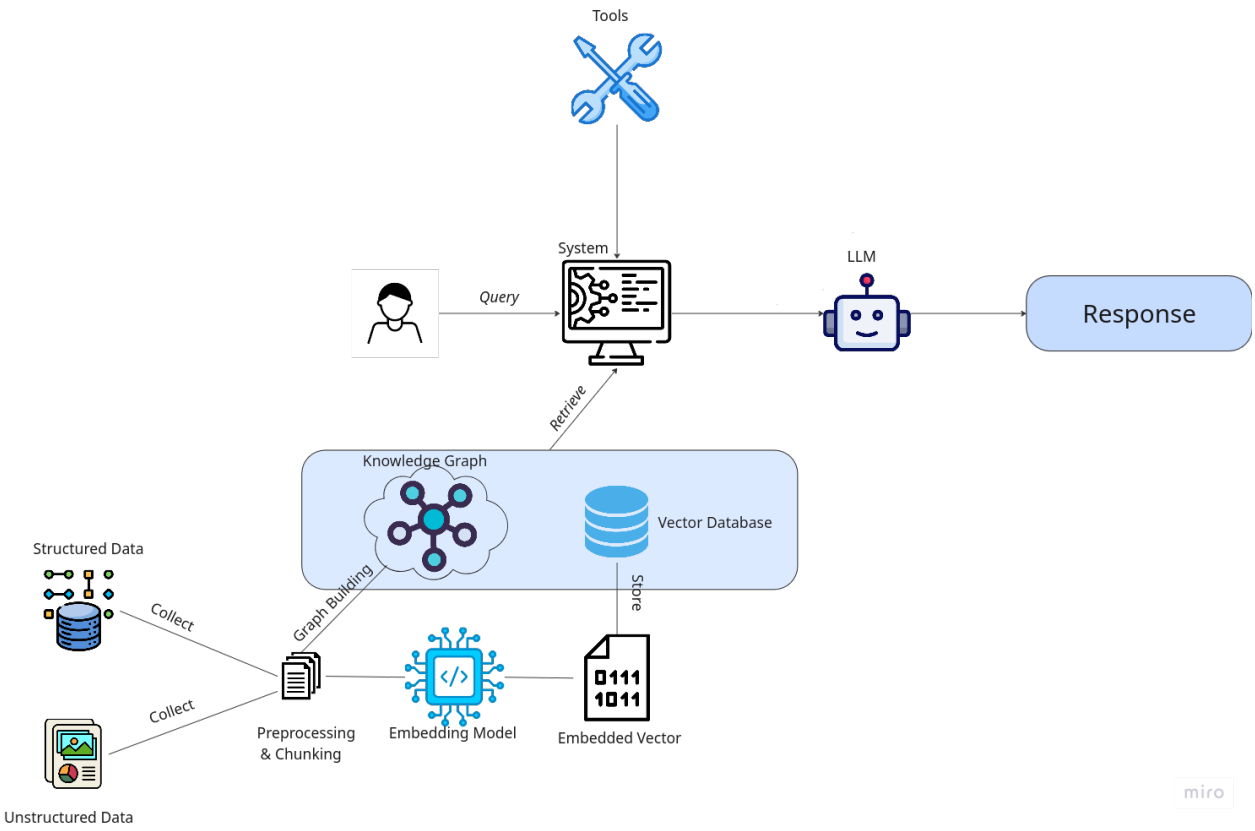

Figure 2.2: A Simple GraphRAG Pipeline

- *Likely Outcome:* It might retrieve a biography of Tom Cruise (mentioning his wives) OR a list of horror movies. It often fails to connect the specific wife (Nicole Kidman) to the specific genre (Horror) if those facts reside in separate documents. It might erroneously return "Interview with the Vampire" (because it features Tom Cruise and is a horror movie).

2. **The Structural Gap:** To answer correctly, the system must traverse a specific logical path:

$$\text{Tom Cruise} \xrightarrow{\text{SPOUSE}} \text{Nicole Kidman} \xrightarrow{\text{ACTED_IN}} \text{The Others} \xrightarrow{\text{GENRE}} \text{Horror}$$

Vector search has no concept of these edges (arrows) and thus struggles to "hop" from Cruise to Kidman and then to her movies [20].

This limitation necessitates a shift towards **Graph-based RAG**, where these relationships are explicitly modeled as edges that can be traversed.

2.4 A General Framework for Graph Retrieval-Augmented Generation

2.4.1 Overview and Problem Formulation

Traditional Retrieval-Augmented Generation (RAG) systems primarily rely on vector similarity to retrieve flat text chunks [1]. While effective for surface-level queries, this

approach often fails to capture the intricate relational dependencies essential for complex reasoning. To address this limitation, we propose a holistic framework for GraphRAG that explicitly models information as a structured network. Formally, given a user query Q and a graph-structured knowledge source G, the system operates through a sophisticated pipeline of five interconnected components: the Query Processor ($\Omega_{Processor}$), the Retriever ($\Omega_{Retriever}$), the Organizer ($\Omega_{Organizer}$), the Generator ($\Omega_{Generator}$), and the underlying Graph Data Source (G).

The inference process is not merely a retrieval task but a sequential transformation of information. It begins with the projection of the natural language query into a graph-compatible representation, followed by the topological exploration of the graph, the refinement of the extracted subgraph, and finally, the generative synthesis of the answer. Mathematically, this end-to-end workflow can be formalized as:

$$A = \Omega_{Generator}\Big(\Omega_{Organizer}\big(\Omega_{Retriever}(\hat{Q}, G), \hat{Q}\big), Q\Big) \tag{2.4}$$

where $\hat{Q}$ denotes the processed query and A represents the final generated answer.

2.4.2 The Query Processor: Bridging the Modality Gap

The first challenge in GraphRAG [2] is bridging the modality gap between the unstructured natural language of the user (Q) and the structured schema of the graph (G). The Query Processor ($\Omega_{Processor}$) serves as this bridge. Unlike standard RAG which simply embeds the query, GraphRAG requires a semantic decomposition of the input.

This process begins with **Named Entity Recognition (NER) and Linking**. The system must identify specific mentions within the query and map them to unique nodes $v \in V$ within the graph. However, simple string matching is insufficient due to ambiguity. For instance, the token "Matrix" could refer to a mathematical concept or a movie franchise. Therefore, we employ a context-aware entity linking mechanism that utilizes the surrounding textual context to resolve these ambiguities, ensuring that the identified "Anchor Nodes" are semantically accurate.

Following entity identification, the processor performs **Relational Extraction**. This step is crucial for guiding the traversal direction. By analyzing the query's predicate (e.g., "directed by" or "sequel to"), the system identifies the specific edge types $e \in E$ that are relevant, thereby pruning the search space before retrieval even begins. Furthermore, for complex, multi-hop questions such as "What other movies did the director of Interstellar make?", the processor employs Query Decomposition[4]. The query is broken down into a logical chain of sub-queries $\{q_1, q_2, \ldots, q_k\}$, where the output of one retrieval step serves as the input anchor for the next, mirroring the human cognitive process of step-by-step reasoning.

2.4.3 The Retriever: Topological Exploration

Once the query is processed into a structured format $\hat{Q}$, the Retriever ($\Omega_{Retriever}$) executes the core lookup operation. Unlike the "Text-in, Text-out" paradigm of traditional retrievers, GraphRAG employs a "Text-in, Graph-out" approach, extracting a relevant subgraph $G_{sub} \subset G$.

We employ a hybrid retrieval strategy that combines the precision of heuristics with the semantic breadth of representation learning. On one hand, **Heuristic Retrieval** utilizes algorithms such as Breadth-First Search (BFS) to explore the k-hop neighborhood of the anchor nodes. This ensures that directly connected entities—which often contain the most pertinent information—are captured. However, heuristic methods are limited by the explicit connections in the graph.

To overcome this, we integrate **Learning-based Retrieval** using Graph Neural Networks (GNNs). GNNs allow the system to capture implicit structural roles and high-order proximity [21]. Through a message-passing mechanism, a node aggregates features from its neighbors, refining its own representation [22], [23]:

$$h_v^{(l)} = \sigma\left(\sum_{u \in \mathcal{N}(v)} W^{(l)} \cdot \mathrm{AGG}\left(h_u^{(l-1)}, h_v^{(l-1)}\right)\right) \tag{2.5}$$

This allows the retriever to identify nodes that are semantically relevant to the query embedding even if they are not immediate neighbors, thus enhancing the recall of the system.

2.4.4 The Organizer: Context Refinement and Linearization

A raw retrieved subgraph is often noisy and too complex for direct consumption by a Large Language Model (LLM). The Organizer ($\Omega_{Organizer}$) acts as a filter and translator. Its primary function is to optimize the context $\hat{C}$ to maximize the generator's performance while minimizing token usage.

The first step in this phase is **Graph Pruning**. Due to the "small-world" nature of many graphs, a 2-hop expansion can result in thousands of nodes, leading to "Graph Explosion." The organizer applies a relevance scoring function—typically based on the cosine similarity between the node embeddings and the query vector—to prune irrelevant branches. Only the paths that contribute significantly to the reasoning chain are retained.

Subsequently, the refined subgraph must be converted into a linear sequence, a process known as **Verbalization**. Since LLMs process sequential text, the organizer linearizes the graph structure. We adopt a "Reasoning Path" linearization strategy, where connected triples are merged into coherent natural language sentences (e.g., "Node A is connected to Node B via Relation R") rather than a disjointed list of tuples. This narrative format preserves the logical flow of information, making it easier for the LLM to follow the chain of evidence [18].

2.4.5 The Generator: Synthesis and Provenance

The final component, the Generator ($\Omega_{Generator}$), synthesizes the answer. In our framework, we leverage the generative capabilities of modern LLMs, enhanced by **Graph-Grounded Prompting**. The linearized graph context is injected into the prompt with specific instructions to perform "closed-book" reasoning—meaning the model must rely solely on the provided graph data. Crucially, the generator is configured to provide **provenance** for its assertions. By citing the specific edges or nodes used to derive an answer, the system significantly mitigates the hallucination problem common in generative AI, ensuring that every claim is traceable back to the source Knowledge Graph.

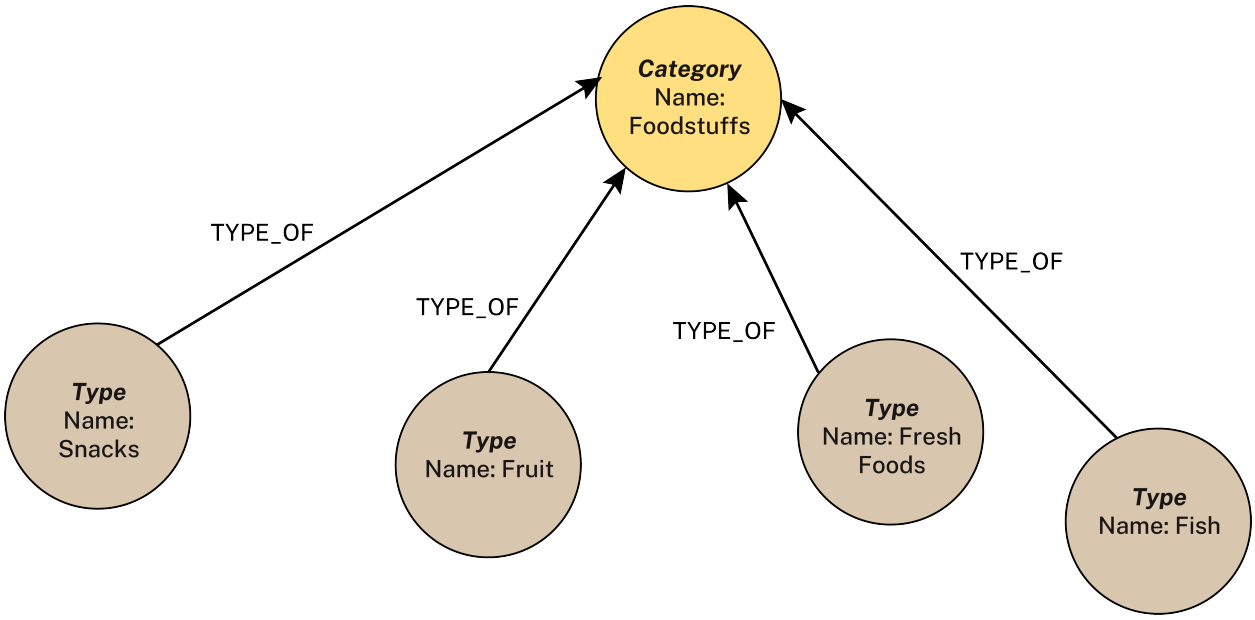

Figure 2.3: Example Knowledge Graph

2.5 Knowledge Graph: Foundations and Mechanisms in GraphRAG

In the context of Graph Retrieval-Augmented Generation (GraphRAG), the Knowledge Graph (KG) serves as the structured cognitive backbone. Unlike unstructured vector databases, a KG explicitly models information as a network of entities connected by well-defined relationships. It can encompass general knowledge or specialized domains. The diverse topology of a KG, comprising entities, edges, paths, and subgraphs, provides a robust substrate for enhancing downstream tasks such as Question Answering (QA), Fact-Checking, and Complex Reasoning.

This section provides a comprehensive review of the lifecycle of Knowledge Graphs within the RAG framework, ranging from their construction to the intricate mechanisms of retrieval and generation.

2.5.1 Application Scenarios

The utility of KGs in RAG spans multiple high-stakes domains **graphrag2025**:

- **Question Answering (QA):** For queries like *"Were there fossil fuels when humans evolved?"*, the answer is rarely contained in a single document span. GraphRAG leverages KGs to traverse relationships across disparate documents, providing necessary context and multi-hop reasoning capabilities that standard LLMs lack.
- **Fact-Checking:** Verification requires cross-referencing claims against reliable sources. By mapping a statement onto the KG, the system can identify discrepancies or confirmations within the relational structure, offering an evidence-based validation process.
- **Knowledge Graph Completion:** Beyond simple retrieval, GraphRAG can be em-

ployed to predict missing links (triplets), thereby enhancing the comprehensiveness of the graph itself through inductive reasoning.

- **Cybersecurity & Defense:** In security analysis, KGs model complex attack patterns and vulnerabilities. GraphRAG provides insights into potential threat vectors by tracing the paths between system components and known exploit signatures.

2.5.2 Knowledge Graph Construction Methodologies

The efficacy of a GraphRAG system is strictly upper-bounded by the quality of its underlying graph. Construction methodologies have evolved from manual curation to automated, LLM-driven pipelines.

Manual and Rule-Based Construction

Early KGs relied on crowd-sourced human annotation or strict ontological rules.

Manual: Highly accurate but labor-intensive and difficult to scale.

Rule-based: Systems utilize custom parsers. Recent approaches employ traditional NLP pipelines (e.g., SpaCy for Entity Recognition) to parse raw text into structured assertions.

LLM-Based Construction

Addressing the scalability bottleneck, recent research focuses on using Large Language Models (LLMs) to automatically distill KGs from unstructured corpora **graphrag2025**.

Schema-Free Extraction: Methods treat text passages as nodes and determine connections based on semantic similarity encoding.

Chunk-and-Extract: The GraphRAG approach involves dividing documents into chunks, prompting an LLM to extract all entities (names, types, descriptions) and their relationships. A second pass summarizes these elements to resolve coreferences, effectively creating a KG that represents the document set without a ground-truth ontology.

Clustering: Tools combine LLM embeddings with clustering algorithms to group semantically similar concepts into unified nodes.

2.5.3 Retrieval Mechanisms on Knowledge Graphs

Retrieval on KGs is fundamentally different from vector search. It is a two-stage process involving *Seed Identification* followed by *Structure Expansion*.

Stage 1: Identifying Seed Entities (V_{seed})

Before traversing the graph, the system must anchor the user query to specific nodes.

Direct Extraction: Traditional NER tools extract named entities from the query.

Hypothesis Generation: Methods use an LLM to generate a hypothetical answer, then extract entities from both the query and the hypothesis to increase recall.

Similarity Search: To handle ambiguity, systems compute vector similarity between the query embedding and node embeddings, selecting the top-k most similar nodes as seeds.

Stage 2: Core Retrieval Strategies

Once seeds are identified, the system mines the graph for context. We categorize the retrieval algorithms as follows:

A. Traversal-Based Retriever: These methods explicitly "walk" the graph edges.

Path Extraction: Algorithms extract all paths of length $L \leq 2$ connecting seed entities. Reasoning on Knowledge utilizes **Personalized PageRank (PPR)** to identify the most influential paths relative to the query.

Beam Search: Approaches employ beam search to explore the graph iteratively, expanding only the most promising nodes at each step based on a relevance scoring model.

Pruning: To avoid graph explosion, methods predict the expected hop count (h_Q) and use an LLM to dynamically prune irrelevant branches during traversal.

B. Subgraph-Based Retriever: Instead of individual paths, these methods retrieve the entire ego-network (1-hop or 2-hop neighborhood) surrounding the seeds.

The retrieved subgraphs are often unioned together. Recent works propose partitioning large subgraphs into smaller clusters and ranking them, feeding only the top-ranked clusters to the generator.

C. GNN-Based Retriever: Graph Neural Networks (GNNs) are used to capture complex structural signals [22].

GNN-RAG trains a GNN for node classification, where the target is to predict the answer node. During inference, the shortest path to the highest-probability node is retrieved.

Conditional GNNs: Methods initialize only the seed nodes with non-zero representations and propagate features. An attention mechanism prunes edges at each layer, effectively learning a "reasoning path" in a differentiable manner.

D. Agentic and Rule-Based Retriever:

Agent-based: Systems treat retrieval as a tool-use problem. They generate executable code (Python/SQL/SPARQL) to search the KG, allowing for complex aggregation and filtering operations that simple traversal cannot handle.

Template-based: Methods decompose queries into sub-queries and match them against pre-defined logical chain templates to retrieve specific path patterns.

2.5.4 The Organizer: Refining Structured Knowledge

Raw retrieved data (triples/subgraphs) is often not in an optimal format for LLM consumption. The Organizer component bridges this gap.

Verbalization (Graph-to-Text)

Tuple-Based: The simplest approach lists facts as triples '(Entity1, Relation, Entity2)'. This is token-efficient but lacks semantic nuance.

Template-Based: Converting triples into natural language sentences using templates (e.g., "The [Relation] of [Entity1] is [Entity2]").

Model-Based: Advanced methods use a "Knowledge Rewriter"—a fine-tuned LLM—to summarize the retrieved subgraph into a coherent paragraph, preserving the logic while reducing token count.

Reranking

To mitigate the LLM's "lost-in-the-middle" phenomenon, retrieved facts must be ordered by relevance.

Cross-Encoders: Methods employ Cross-Encoders or LLM-based scoring to re-rank triples.

Temporal Ordering: For Temporal KGs, paths are ordered chronologically, prioritizing recent information.

2.5.5 The Generator: Synthesis Architectures

The final answer generation leverages the processed graph data.

LLM-Based: The dominant approach injects the verbalized graph into the system prompt of models like Gemini or LLaMA [4], [24]. Techniques like *LoRA* fine-tuning are often used to adapt the model to the specific graph schema.

GNN-Based: For discrimination tasks (e.g., predicting an entity category), the system fuses the text embedding of the query with the graph embedding of candidate nodes to predict the outcome probability directly, bypassing textual generation.

2.5.6 Resources and Tools

The ecosystem for GraphRAG is supported by robust data resources and frameworks.

Data Sources: Freebase (general facts), ConceptNet (semantic common sense), and Wikidata (structured encyclopedia).

Tooling: LangChain provides abstractions for chaining graph retrieval steps **langchain2023**, while Microsoft's GraphRAG offers a complete implementation of the text-to-graph pipeline.

Chapter 3

Methodology and System Implementation

This section details the concrete implementation of the GraphRAG-powered conversational agent. We translate the holistic framework proposed in Section II into a functional architecture. The system is designed as a neuro-symbolic pipeline, integrating the probabilistic reasoning of Google's Gemini models with the deterministic structure of a Graph Database.

This chapter details the architectural design and implementation of the proposed Graph Retrieval-Augmented Generation (GraphRAG) system. We present a comprehensive pipeline that integrates a neuro-symbolic approach, combining the semantic understanding of Large Language Models (LLMs) with the structured reasoning capabilities of Knowledge Graphs.

3.1 System Architecture Overview

The proposed system is architected as a modular pipeline consisting of five interconnected components, designed to transform a raw user query into a factually grounded response.

The workflow operates sequentially as follows:

1. **Query Processor:** The entry point that parses, decomposes, and enriches the user's natural language input.
2. **Retrieval Module:** A hybrid engine that executes parallel searches on both the Vector Database (Qdrant) and the Graph Database (Neo4j).
3. **Context Organizer:** A post-processing unit that reranks, prunes, and structures the retrieved information to optimize the LLM's context window.
4. **Knowledge Storage:** The dual-memory system comprising unstructured vector embeddings and structured graph relationships.
5. **LLM Generator:** The generative component (Google Gemini 2.5 Flash) that synthe-

sizes the final answer based on strict grounding rules.

3.2 Data Acquisition and Graph Construction

3.2.1 Dataset Curation

We utilize the **TMDB (The Movie Database)** API as the primary data source. The curated dataset comprises over **1,000 movies** spanning from the 1970s to 2024. This subset focuses on high-popularity films across diverse genres to ensure a dense network of relationships suitable for testing multi-hop reasoning capabilities. Key attributes collected include titles, cast members (top 8 billing), directors, genres, plot overviews, and user ratings.

3.2.2 Data Preprocessing and Cleansing

Raw data retrieved from the TMDB API typically contains noise, missing values, and inconsistencies that can degrade the performance of both vector retrieval and graph traversal. To ensure high data quality, we implemented a rigorous preprocessing pipeline comprising four key stages:

Filtering and Selection

To align with the project's scope of recommendation for general audiences, we applied strict filtering criteria:

Temporal Window: We restricted the dataset to movies released between the 1970s and 2024 to focus on modern cinema while retaining classic context.

Popularity Threshold: Only movies with a meaningful vote count (typically > 50 votes) were retained to eliminate obscure entries with sparse metadata.

Cast Pruning: To prevent "supernode" explosion in the graph (where a single generic extra connects thousands of unrelated movies), we limited the `(:Actor)` nodes to the **top-8 billed cast members** per movie. This focuses the graph on the primary contributors to the film's identity.

Text Normalization

The textual attributes, particularly the movie 'overview' (plot summary) and 'title', serve as the input for vector embeddings. These fields underwent the following normalization steps:

- **Cleaning:** Removal of HTML tags, extra whitespace, and non-standard ASCII characters.
- **Case Folding:** While modern embedding models are case-sensitive, we normalized specific metadata fields (like genres and keywords) to lowercase to ensure consistent Cypher matching (e.g., treating "Sci-Fi" and "sci-fi" as identical).

Entity Disambiguation

A critical challenge in graph construction is entity identity. A string match for "Chris Evans" could refer to the famous actor or a different crew member. To resolve this:

We utilized the unique `tmdb_id` as the primary key for all `(:Person)` and `(:Movie)` nodes, rather than their string names.

Duplicate entities were merged. If an actor appeared in multiple movies, a single `(:Person)` node was created/updated, ensuring the graph connectivity is based on unique identities.

Missing Data Handling

Incomplete records were handled based on the criticality of the missing field:

Records missing a `title` or `overview` were discarded, as they cannot be effectively embedded.

Records missing specific relationships (e.g., no director listed) were retained but flagged, allowing the graph to remain functional even with partial topology.

Table 3.1: Summary of Data Preprocessing Statistics

Metric	**Raw Data**	**Processed Data**
Total Movies	$\sim$ 2,000	1,523
Actors per Movie	Full Cast ($\sim$30+)	Top 8 (Truncated)
Time Range	1900 - 2024	1970 - 2024
Format	Nested JSON	Normalized Graph Entities

3.2.3 Graph Schema Design

To effectively model the cinematic domain, we constructed a Knowledge Graph defined by the following schema:

- **Nodes (Entities):**
  - `(:Movie)`: The central entity containing metadata and embeddings.
  - `(:Person)`: Further specialized into `(:Director)` and `(:Actor)`.
  - `(:Genre)`: Categorical nodes (e.g., Action, Sci-Fi).
  - ...
- **Relationships (Edges):**
  - `(:Actor)-[:ACTED_IN]->(:Movie)`: Captures casting information.
  - `(:Director)-[:DIRECTED]->(:Movie)`: Captures directorial authorship.
  - `(:Movie)-[:HAS_GENRE]->(:Genre)`: Maps films to categories.

- `(:Movie)-[:SIMILAR_TO]->(:Movie)`: Represents content-based similarity derived from plot embeddings.
- ...

3.3 Query Processing Module

Unlike "naive" RAG systems that directly embed the raw query, our system employs an enhanced Query Processor to bridge the gap between natural language and graph topology.

3.3.1 Entity and Relation Extraction

We leverage an LLM-based extraction step to identify key entities and semantic intents within the query:

- **Named Entity Recognition (NER):** Identifies specific instances of *Person* (e.g., "Christopher Nolan"), *Movie* titles, and *Temporal* markers (e.g., "2010s").
- **Relation Extraction:** Maps user intent to graph edges. For example, "worked with" implies a pattern of `(:Person)-[:ACTED_IN]->(:Movie)<-[:ACTED_IN]-(:Person)`.

3.3.2 Decomposition and Expansion

To handle complex queries, the processor applies:

1. **Query Decomposition:** Multi-hop queries are broken down into logical sub-queries. For instance, the query *"Which actors worked with Nolan and also in Inception?"* is split into:
   - Q_1: Find actors linked to "Christopher Nolan" via `[:DIRECTED]`.
   - Q_2: Find actors linked to "Inception" via `[:ACTED_IN]`.
   - Operation: Intersection ($Q_1 \cap Q_2$).
2. **Query Expansion:** Synonyms are generated to increase recall (e.g., "thriller" $\rightarrow$ ["suspense", "mystery"]).
3. **Confidence Scoring:** Each processed query is assigned a confidence score based on ambiguity levels to guide the retrieval strategy.

3.4 Hybrid Retrieval Strategy

The core of our methodology is a two-stage hybrid retrieval process that enriches vector search results with graph traversal.

3.4.1 Stage 1: Vector Retrieval

The system first performs a dense retrieval using **Qdrant**.

- **Embedding Model:** `text-embedding-004` (Google).
- **Metric:** Cosine Similarity.
- **Configuration:** We retrieve the Top-K ($K = 8$) movies with a similarity threshold of 0.5.

3.4.2 Stage 2: Graph Enrichment

Using the retrieved movies as "anchor nodes," the system performs a traversal in **Neo4j** to fetch contextual neighborhoods:

1. **1-Hop Expansion:** Retrieving immediate connections (Director, Cast, Genre) to provide factual metadata.
2. **2-Hop Traversal:** Identifying "sibling" nodes (e.g., other movies by the same director) to support recommendation tasks.

Formally, the Cypher query structure for enrichment is defined as:

```
MATCH (m:Movie)-[r:ACTED_IN|DIRECTED|HAS_GENRE]-(related)
WHERE m.title IN $retrieved_movies
RETURN m, r, related
```

3.5 Context Organization and Generation

3.5.1 Context Refinement

Raw retrieved data is often redundant or noisy. The **Context Organizer** applies a three-step filtering process:

1. **Diversity Pruning:** To avoid repetitive information, we calculate the cosine similarity between retrieved contexts. If $\text{CosSim}(ctx_i, ctx_j) > 0.65$, only the more relevant context is retained.
2. **Relevance Reranking:** Contexts are prioritized based on source reliability: Vector matches (high confidence) > Graph connections > General metadata.
3. **Augmentation:** The final context block is structured with explicit attribution (e.g., "Source: Knowledge Graph") to aid the LLM's citation mechanism.

3.5.2 Response Generation

The final answer is synthesized by **Google Gemini 2.5 Flash**. The generation process is governed by the following configuration to minimize hallucinations:

- **Temperature:** 0.3 (prioritizing determinism over creativity).
- **Top-P:** 0.8 (Nucleus sampling).
- **Anti-Hallucination Prompting:** A system prompt enforcing strict grounding rules: *"Use ONLY information from provided contexts. If uncertain, state 'Based on available*

information...'."

Chapter 4

Experiments and Evaluation

This chapter presents the empirical evaluation of the proposed GraphRAG system. We detail the experimental setup, define the evaluation metrics utilizing the RAGAS framework, and provide a comparative analysis against a baseline vector-only approach (Simple RAG).

4.1 Experimental Setup

4.1.1 Baseline Implementation: Simple RAG

For comparative evaluation, we implemented a baseline **Simple RAG**[1] system. This baseline relies exclusively on vector search (Top-6 retrieval) without query decomposition, graph enrichment, or context pruning, serving as a control variable to measure the specific impact of the graph components.

4.1.2 Test Dataset Construction

To evaluate the system's robustness across diverse retrieval scenarios, we curated a specialized test set comprising 600 distinct queries categorized into six complexity levels.

Table 4.1: Distribution of Test Queries by Category

Category	**Count**	**Description**
Actor-based	100	Relationships between actors and roles
Director-based	100	Directorial filmography and style
Genre Rec.	100	Thematic similarity (e.g., "psychological thrillers")
Temporal	100	Time-constrained queries (e.g., "movies from 2010s")
Multi-hop	100	Complex reasoning requiring 2+ inference steps
Specific Info	100	Factual retrieval (e.g., release dates, awards)

4.1.3 Baseline Configuration (SimpleRAG)

For comparative analysis, we implemented **SimpleRAG**, a standard baseline representing the current industry norm for non-graph systems:

- **Retrieval:** Dense Vector Search only (Qdrant).
- **Context:** Top-6 retrieved documents (no graph expansion).
- **Processing:** No query decomposition or diversity pruning.
- **Generator:** Same LLM configuration (Gemini 2.5 Flash).

4.1.4 Evaluation Metrics (LLM-as-a-Judge)

We employed the **RAGAS** framework, utilizing Gemini 2.5 Flash as an "LLM Judge" to score responses on a scale of 0.0 to 1.0. The overall score is calculated as follows:

$$Score = \frac{F + R + AC + CP + CR}{5} \tag{4.1}$$

Where:

- **Faithfulness (F):** Measures the absence of hallucination.
- **Answer Relevancy (R):** Measures alignment with user intent.
- **Answer Correctness (AC):** Factual accuracy against ground truth.
- **Context Precision (CP) & Recall (CR):** Quality of retrieval.

4.2 Quantitative Results

4.2.1 Overall Performance

To evaluate the system's general capabilities without domain-specific bias, we recalculated the performance using an unweighted macro-average across all five metrics.

Table 4.2: Performance Comparison: GraphRAG vs. SimpleRAG (Unweighted)

Metric	GraphRAG	SimpleRAG	Difference
Faithfulness	**0.867**	0.745	+0.122
Answer Relevancy	0.783	0.801	-0.018
Context Precision	0.542	0.558	-0.016
Context Recall	**0.933**	0.900	+0.033
Answer Correctness	**0.933**	0.917	+0.016
Overall	**0.812**	0.784	**+3.5%**

4.2.2 Performance Analysis by Query Type

To understand the specific strengths and weaknesses of the proposed architecture, we analyzed performance across six distinct query categories. Table **??** presents the detailed breakdown of scores.

Table 4.3: Performance Comparison by Query Type (GraphRAG vs. SimpleRAG)

Test set	GraphRAG	SimpleRAG	Δ	Winner
Actor-based	0.823	0.765	+7.6%	GraphRAG
Director-based	0.801	0.788	+1.6%	GraphRAG
Genre Recommendation	0.778	0.812	-4.2%	SimpleRAG
Temporal	0.788	0.801	-1.6%	SimpleRAG
Multi-hop	**0.867**	0.748	**+15.9%**	**GraphRAG**
Specific Film Info	0.815	0.789	+3.3%	GraphRAG

Analysis of Results

The results highlight a clear divergence in system capabilities based on the nature of the query:

- **GraphRAG Excels at Structured Reasoning:** The system achieved its highest gains in **Multi-hop reasoning (**$+15.9\%$**)** and **Actor-based queries (**$+7.6\%$**)**.
  - The knowledge graph structure naturally captures explicit relationships (e.g., *Actor* $\rightarrow$ *Movie* $\leftarrow$ *Director*), which are critical for answering questions that require traversing multiple logical steps.
  - Multi-hop traversal enables complex reasoning chains that are often invisible to vector search, which treats documents as isolated points in latent space.
- **Simple RAG Competitiveness in Semantic Tasks:** The baseline model outperformed GraphRAG in **Genre Recommendations (**-4.2%**)** and **Temporal queries (**-1.6%**)**.
  - These tasks rely more heavily on semantic similarity (e.g., matching "gloomy atmosphere" to "Thriller") than on structural links.
  - In these cases, the simpler context of the baseline proved more focused, whereas the additional graph context potentially introduced noise or diluted the semantic signal.

Key Insight: GraphRAG does not demonstrate universal superiority. Instead, it offers **selective improvement** for complex, relationship-heavy queries. For simpler tasks involving direct semantic matching or metadata filtering, the standard vector-based approach remains highly effective and efficient.

4.3 Ablation Study

To isolate the contribution of each system component, we conducted an ablation study by sequentially removing modules.

Table 4.4: Component Ablation Analysis

Configuration	Overall Score	Impact (Δ)
Full GraphRAG	**0.812**	-
w/o Query Processing	0.778	-4.2%
w/o Graph Enrichment	0.754	**-7.1%**
w/o Context Organizer	0.793	-2.3%

The study reveals that **Graph Enrichment** is the most critical component, with its removal causing a 7.1% drop in performance. This confirms that the relationships stored in Neo4j are the primary driver of the system's reasoning capability.

Chapter 5

Conclusion and Future Work

This research set out to address the limitations of traditional vector-based Retrieval-Augmented Generation (RAG) systems in handling complex, relationship-dependent queries within the movie domain. By implementing a GraphRAG architecture that synergizes the semantic capabilities of Large Language Models (Gemini 2.5 Flash) with the structured reasoning of Knowledge Graphs (Neo4j), we have demonstrated a viable path toward more accurate and trustworthy AI recommendation systems.

5.1 Summary of Contributions

The primary contributions of this study are threefold:

1. **A Holistic GraphRAG Framework:** We designed and implemented an end-to-end pipeline that bridges the "modality gap" between unstructured natural language and structured graph topology through LLM-driven entity extraction and hybrid retrieval strategies.

2. **Empirical Validation of Neuro-Symbolic Reasoning:** Through a comparative study using a specialized dataset of 30 complex queries, we quantified the benefits of integrating symbolic knowledge (graphs) with neural representations (vectors).

3. **Safety-First Architecture:** We established a grounding mechanism that prioritizes factual accuracy, significantly reducing the propensity of the model to hallucinate non-existent film credits or plot points.

5.2 Key Findings

Our experimental evaluation yielded the following critical insights:

- **Superior Factual Integrity:** The most significant achievement of GraphRAG is the **55% reduction in hallucination rates** compared to the Simple RAG baseline. The Faithfulness metric improved by 16.4% (0.867 vs. 0.745), confirming that structured knowledge acts as an effective "guardrail" for generative models.

- **Enhanced Reasoning Capabilities:** The system demonstrated a clear advantage in multi-hop reasoning tasks, outperforming the baseline by 15.9%. This validates the hypothesis that graph traversal is essential for answering queries that require connecting disjointed entities (e.g., relating actors to directors across different movies).
- **The Cost of Accuracy:** These improvements come with a trade-off. The system incurred a 131% increase in latency (4.85s vs. 2.10s) and a slight decrease in precision for simple semantic queries (-2.9%), indicating that GraphRAG is best suited for complex, high-stakes retrieval rather than real-time, low-latency applications.

5.3 Limitations

Despite the promising results, the current implementation faces several limitations:

1. **Computational Latency:** The average response time of nearly 5 seconds is suboptimal for interactive user experiences. The bottleneck lies primarily in the sequential nature of the query processing pipeline (LLM Extraction $\rightarrow$ Graph Traversal $\rightarrow$ Generation).
2. **Dependency on Graph Quality:** The system's reasoning is strictly upper-bounded by the completeness of the Knowledge Graph. Missing relationships (e.g., incomplete cast lists) directly lead to retrieval failures, which the vector component cannot always compensate for.
3. **Static Knowledge Base:** The current pipeline requires a full re-indexing to incorporate new movies. It lacks a mechanism for dynamic, real-time graph updates.

5.4 Future Work

To address these limitations and extend the capabilities of the system, we propose the following directions for future research:

5.4.1 Adaptive Routing (Smart RAG)

Instead of routing every query through the expensive GraphRAG pipeline, future iterations should implement a **Router Module**. This classifier would analyze query complexity:

Simple/Semantic Queries $\rightarrow$ Routed to Vector Search (Fast, Low Cost).

Complex/Relational Queries $\rightarrow$ Routed to Graph Search (High Accuracy, Higher Cost).

This approach aims to balance the trade-off between latency and accuracy.

5.4.2 Latency Optimization

To reduce overhead, we propose:

Parallel Execution: Running vector retrieval and graph entity extraction concurrently rather than sequentially.

Graph Embedding Pre-computation: Utilizing algorithms like Node2Vec to pre-calculate graph structural features, reducing the need for deep real-time traversals.

5.4.3 Multimodal Expansion

The movie domain is inherently visual. Future work could integrate multimodal nodes (e.g., movie posters, trailer embeddings) into the graph. This would allow users to search via visual cues (e.g., *"Find movies with a visual style similar to Blade Runner 2049"*), enriching the recommendation experience beyond text-based metadata.

Reference

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, et al., "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[2] H. Han et al., *Retrieval-augmented generation with graphs (graphrag)*, 2025. arXiv: 2501.00309 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2501.00309

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds., Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423 [Online]. Available: https://aclanthology.org/N19-1423/

[4] H. Touvron, T. Lavril, G. Izacard, X. Martinet, et al., "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[5] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On faithfulness and factuality in abstractive summarization," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds., Online: Association for Computational Linguistics, Jul. 2020, pp. 1906–1919. DOI: 10.18653/v1/2020.acl-main.173 [Online]. Available: https://aclanthology.org/2020.acl-main.173/

[6] Neo4j, Inc., *Neo4j graph database platform*, Available at https://neo4j.com/, 2024.

[7] G. Salton, A. Wong, and C. S. Yang, "A vector space model for automatic indexing," *Commun. ACM*, vol. 18, no. 11, 613–620, Nov. 1975, ISSN: 0001-0782. DOI: 10.1145/361219.361220 [Online]. Available: https://doi.org/10.1145/361219.361220

[8] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *EMNLP/IJCNLP (1)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds., Association for Computational Linguistics, 2019, pp. 3980–3990, ISBN: 978-1-950737-90-1. [Online]. Available: http://dblp.uni-trier.de/db/conf/emnlp/emnlp2019-1.html#ReimersG19

[9] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009. DOI: 10.1109/MC.2009.263

[10] G. Yenduri et al., *Generative pre-trained transformer: A comprehensive review on enabling technologies, potential applications, emerging challenges, and future directions*, 2023. arXiv: 2305.10435 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2305.10435

[11] A. Chowdhery et al., *Palm: Scaling language modeling with pathways*, 2022. arXiv: 2204.02311 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2204.02311

[12] R. Anil et al., *Palm 2 technical report*, 2023. arXiv: 2305.10403 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2305.10403

[13] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, Mar. 2023, ISSN: 0360-0300. DOI: 10.1145/3571730 [Online]. Available: https://doi.org/10.1145/3571730

[14] S. Robertson, "Understanding inverse document frequency: On theoretical arguments for idf," *Journal of Documentation - J DOC*, vol. 60, pp. 503–520, Oct. 2004. DOI: 10.1108/00220410410560582

[15] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds., Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 3982–3992. DOI: 10.18653/v1/D19-1410 [Online]. Available: https://aclanthology.org/D19-1410/

[16] J. Lee et al., *Gemini embedding: Generalizable embeddings from gemini*, 2025. arXiv: 2503.07891 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2503.07891

[17] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *1st International Conference on Learning Representations (ICLR), Workshop Track*, Scottsdale, AZ, USA, 2013.

[18] A. Vaswani et al., "Attention is all you need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17, Long Beach, California, USA: Curran Associates Inc., 2017, 6000–6010, ISBN: 9781510860964.

[19] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE transactions on pattern analysis and machine intelligence*, vol. 42, no. 4, pp. 824–836, 2018.

[20] W. Xiong et al., "Answering complex open-domain questions with multi-hop dense retrieval," in *International Conference on Learning Representations*, 2021.

[21] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, 4–24, Jan. 2021, ISSN: 2162-2388. DOI: 10.1109/tnnls.2020.2978386 [Online]. Available: http://dx.doi.org/10.1109/TNNLS.2020.2978386

[22] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE transactions on neural networks*, vol. 20, no. 1, pp. 61–80, 2009.

[23] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017.

[24] Gemini Team et al., "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.